PROJECT_REPORT_AND_TESTS.md

项目目标

本项目旨在完成一个基于 MongoDB 的在线问卷系统，实现课程要求中的问卷创建、编辑、发布、填写、跳转、校验与统计等核心能力。系统需要同时满足两类用户场景：一类是问卷拥有者在后台完成问卷管理与结果查看；另一类是填写者通过公开链接进行作答。

第一阶段的重点不在于页面视觉复杂度，而在于数据契约的明确、后端逻辑的闭环以及测试材料的完整。项目最终需要提交可运行代码、数据库结构、接口说明、关键逻辑说明、测试用例与项目完成报告，因此文档组织必须能够直接映射评分项。

系统设计

系统采用前后端分离结构。前端使用 `React + Vite + TypeScript` ，后端使用 `Node.js + Express + TypeScript` ，数据库采用 `MongoDB + Mongoose` 。

前端包括两部分。第一部分是管理后台，用于注册登录、问卷列表查看、题目编辑、发布与统计查看。第二部分是填写端，用于通过问卷链接读取公开定义、执行作答交互并提交答案。

后端按照 Controller、Service、Model、Route、Middleware 进行分层。Controller 负责请求响应封装，Service 负责核心业务逻辑，Model 负责 MongoDB 数据契约，Route 负责接口组织，Middleware 负责鉴权与统一异常处理。

系统设计上最重要的原则是将“体验层路径推演”和“后端真实性校验”彻底分开。前端可以做分页切换与即时跳转体验，但所有实际提交必须以后端重演结果为准，这样才能避免伪造请求污染数据库。

MongoDB设计

数据库包含 `Users` 、 `Surveys` 、 `Responses` 三个核心集合。

`Users` 用于保存系统用户账号信息，记录用户名、密码哈希与创建时间。

`Surveys` 用于保存一份问卷的完整定义。每个问卷文档内嵌题目数组，题目内部再嵌套选项、校验规则与跳转规则。之所以采用这种设计，是因为问卷在业务上天然是一个整体：渲染时需要整份取出，校验时也需要整份分析。如果拆成关系型多表，不但读取链路更长，更新过程也会更繁琐。

`Responses` 用于保存答卷结果。答卷内记录 `surveyId` 、 `respondentId` 、提交时间以及答案数组。答案中的题目标识全部使用稳定的 `questionId` ，不使用数组下标表达业务身份，从而保证结构变动时不会破坏历史数据含义。

MongoDB 文档结构还特别适合本项目的统计需求。答卷中的 `answers` 数组可以直接通过 `$unwind` 展开，再使用 `$group` 、 `$avg` 、 `$push` 等聚合操作计算各类题型的统计结果，既减少了表关联，又让统计代码保持清晰。

跳转设计

系统中的跳转规则不是写死在前端组件内部，而是作为问卷定义的一部分保存在每道题的 `logicRules` 与 `defaultNextQuestionId` 字段中。也就是说，跳转逻辑属于数据，而不是界面行为。

每道题可以配置若干条条件规则，例如某个选项命中后跳到指定题目，或在数字题中根据数值区间跳转到不同分支。当没有任何规则命中时，系统再走默认跳转。这样设计有两个直接好处。第一，问卷逻辑可以通过数据修改而不是改代码来实现。第二，前后端都能基于同一份问卷定义做路径推导，不容易出现语义分裂。

为了防止错误跳转进入运行期，系统在创建、更新和发布问卷时都会提前校验跳转图是否存在自循环、图循环和不可达题目。只有通过这些结构性检查后，问卷才允许进入发布态。

校验设计

校验分为结构校验和提交校验两部分。

结构校验发生在创建草稿、更新草稿和发布问卷时，重点检查题目 ID 是否重复、题目顺序是否冲突、选项 ID 是否重复、校验区间是否自相矛盾、跳转目标是否存在，以及跳转图是否包含循环和不可达节点。

提交校验发生在填写者提交答卷时，内容更细。系统会先校验请求体结构，再检查问卷是否处于可填写状态、是否超过截止时间、是否需要登录。随后根据题型分别检查答案合法性，例如单选必须来自合法选项，多选数量不能越界，文本长度必须落在允许区间，数字题必须满足最小值、最大值和整数约束。

最重要的一步是路径重演校验。服务端不会相信前端送来的答案顺序，而是根据题目定义重新计算从入口题到 `END` 的真实路径。只有位于这条路径上的答案才允许入库，其余答案全部视为非法跳题或幽灵答案。

AI使用过程

开发过程中使用了大模型工具进行辅助，但项目的结构设计、规则取舍与最终代码落地仍由人工完成。AI 的作用主要体现在初稿生成、方案比选和局部重构建议，而不是替代完整开发过程。

在问卷路径重演与防伪造架构设计阶段，先围绕“如何同时保留前端沉浸式分页体验和后端数据真实性”进行讨论。AI 给出了“前端体验级引擎 + 后端最终裁决”的方向，并为前端状态机和后端重演逻辑提供了初稿。最终人工明确了原则：前端可以预演，后端必须重演，真实提交只能以后端为准。

在 TypeScript 与 API 契约整理阶段，AI 协助完成了请求封装、错误类型抽象、接口字段对齐与前后端联调改造。例如前端从早期 Mock 数据切换到真实 `/render` 和 `/submit` 接口时，AI 给出了基础封装方式，但最终仍需人工以项目已实现接口为基准进行修订，避免出现文档和代码不一致的问题。

在前端页面实现阶段，AI 生成过多个布局与样式初稿，其中部分方案虽然可运行，但存在风格偏离或装饰过度的问题。后续通过人工多轮审查与重写，逐渐收敛出更符合项目要求的实现方式。这部分经历也说明，AI 在提供草案时速度很快，但在审美一致性和细节把控上并不可靠。

在 MongoDB 结构设计与统计管道阶段，AI 对聚合写法和文档模型提供了较明显帮助。例如针对单选、多选、数字与文本题的统计计算，AI 给出了 `$unwind` 、 `$group` 、 `$avg` 、 `$push` 等基础写法；而真正落地时，仍然需要根据本项目的 `Responses.answers` 结构重新组织聚合管道，并按问卷题目拼装最终返回结果。

在测试建设阶段，AI 帮助补齐了集成测试的基础框架，覆盖注册登录、创建发布、合法提交、非法数值、幽灵答案、匿名权限与统计接口等关键路径。但课程要求不仅要有成功路径，也要有防线测试和可读的测试文档，因此最终仍由人工补充手工联调记录、整理测试报告并校对最终提交材料。

整体来看，AI 最有价值的地方在于提高重复性编码、类型整理、聚合初稿和界面骨架搭建的效率；而在架构拍板、契约收敛、校验语义确认、风格判断和最终验收上，仍然必须依赖人工主导。项目的最终成果来自“AI 提供初稿 + 人工审查修正 + 真实联调验证”的协同过程，而不是自动生成后直接提交。

测试结果

测试结果覆盖了创建问卷、添加题目、动态跳转、数值校验、匿名权限控制、问卷提交与统计查询等关键场景。

自动化测试部分使用 Jest 与 `mongodb-memory-server` 执行集成测试，验证了注册登录、创建问卷、发布问卷、读取公开问卷定义、提交合法答卷、拦截非法数值、拦截幽灵答案以及读取统计结果等流程。相关测试均已通过。

本次提交新增了前端回归测试 `tests/frontend-survey-engine.test.ts` ，重点覆盖三类场景：线性默认路径在首题未作答时的总页数稳定性、分支题未作答时停止预计算、以及已回答前缀场景下当前页定位正确。

手工联调部分重点验证真实前后端环境下的填写体验与数据落库情况，包括 `/render` 页面渲染、分页跳转、真实答卷提交流程、统计接口落库验证以及浏览器端稳定性检查。联调结果表明：两题问卷可以从第 1 页正确进入第 2 页并完成提交，数据库 `responses` 计数可增长，统计链路恢复正常。

综合来看，本阶段已完成从用户注册登录到问卷管理、问卷填写、后端校验、答卷入库和统计查看的完整闭环，满足课程第一阶段的主要验收要求。

遇到的问题

第一个问题出现在动态跳转问卷的答案残留上。用户在填写过程中如果先沿某一条路径完成部分作答，随后回到前面修改关键答案，原先那条路径上的后续答案就可能变成无效数据。如果系统只保留前端局部状态，而不做全局重算，就容易把已经不可达的旧答案一起提交出去。

第二个问题出现在问卷编辑器的嵌套状态更新上。题目本身是数组，题目内部还包含选项数组、跳转规则数组和校验对象，在题型切换时尤其容易出现旧字段残留、表单状态混乱或局部更新遗漏。

第三个问题出现在前后端联调阶段。早期部分页面逻辑依赖本地 Mock 数据，切换到真实接口后，需要重新处理统一响应包、错误信息透传、Vite 代理与问卷 ID 的动态读取，否则浏览器端无法顺利走通完整链路。

第四个问题出现在填写端前端状态机。 `replayFromStart` 在首题未作答时过早停止，导致线性问卷总页数显示偏小；同时翻页动画阶段存在状态未及时回收问题，导致“下一页/提交”按钮长期处于禁用状态，表现为提交无响应、统计计数不增长。

第五个问题出现在文档组织方式上。随着开发推进，系统说明、API、测试记录、项目报告和 AI 使用日志逐渐分散到多个文件中，虽然内容本身完整，但不利于最终按评分项直接提交。

如何解决

针对答案残留问题，最终采用“前端体验级路径重算 + 后端全量路径重演”的双重方案。前端在关键答案变动时重新计算当前有效路径，用于更新界面体验；后端在提交时再以问卷定义为依据，从入口题完整重演一遍真实路径，从源头拦截幽灵答案与非法跳题。

针对编辑器状态更新复杂的问题，采用了明确的数据标准化策略。每次修改题目时，都将 `options` 、 `validation` 、 `logicRules` 归一到稳定结构，并在题型切换时主动清理不再适用的字段，减少旧状态污染。

针对联调问题，统一以前后端实际接口契约为基准，补齐前端请求封装、错误处理与开发代理配置，并将填写页改为从真实 URL 中读取问卷 ID，保证 `/render` 与 `/submit` 可以在浏览器环境下直接联通。

针对本次提交暴露的前端状态问题，已进行两项修复。其一，在 `frontend/src/state/survey-engine.ts` 调整路径预演逻辑：首个未作答题仍作为当前可作答页；若该题无 `logicRules` ，则继续沿 `defaultNextQuestionId` 预计算线性路径总页数；遇到分支题后停止预计算，避免错误预判。其二，在 `frontend/src/components/desk-scene.tsx` 修复翻页与提交状态回收： `forward-out` 与 `backward-out` 完成后立即回到 `idle` ，并同步完善目标题目与检索完成标记时机，消除按钮锁死与提交无响应问题。

针对文档碎片化问题，最终按照评分标准重新收拢文档结构，将系统说明、数据库设计、API 说明与关键逻辑整合到 `SYSTEM_AND_API_DESIGN.md` ，将测试材料集中到 `TEST_CASES.md` ，将项目完成报告、测试结果、问题与解决过程、AI 使用过程整合到 `PROJECT_REPORT_AND_TESTS.md` ，并用 `README.md` 保留运行环境与启动说明，从而使提交材料与评分项形成一一对应关系。

仓库地址：https://github.com/woobowen/Data_Management_System